

# **Injured application penetration testing**



2:27



# InjuredAndroid



XSSTEXT

FLAG ONE - LOGIN

FLAG TWO - EXPORTED ACTIVITY

FLAG THREE - RESOURCES

FLAG FOUR - LOGIN 2

FLAG FIVE - EXPORTED BROADCAST RECEIVER

FLAG SIX - LOGIN 3

FLAG SEVEN - SQLITE

required tools

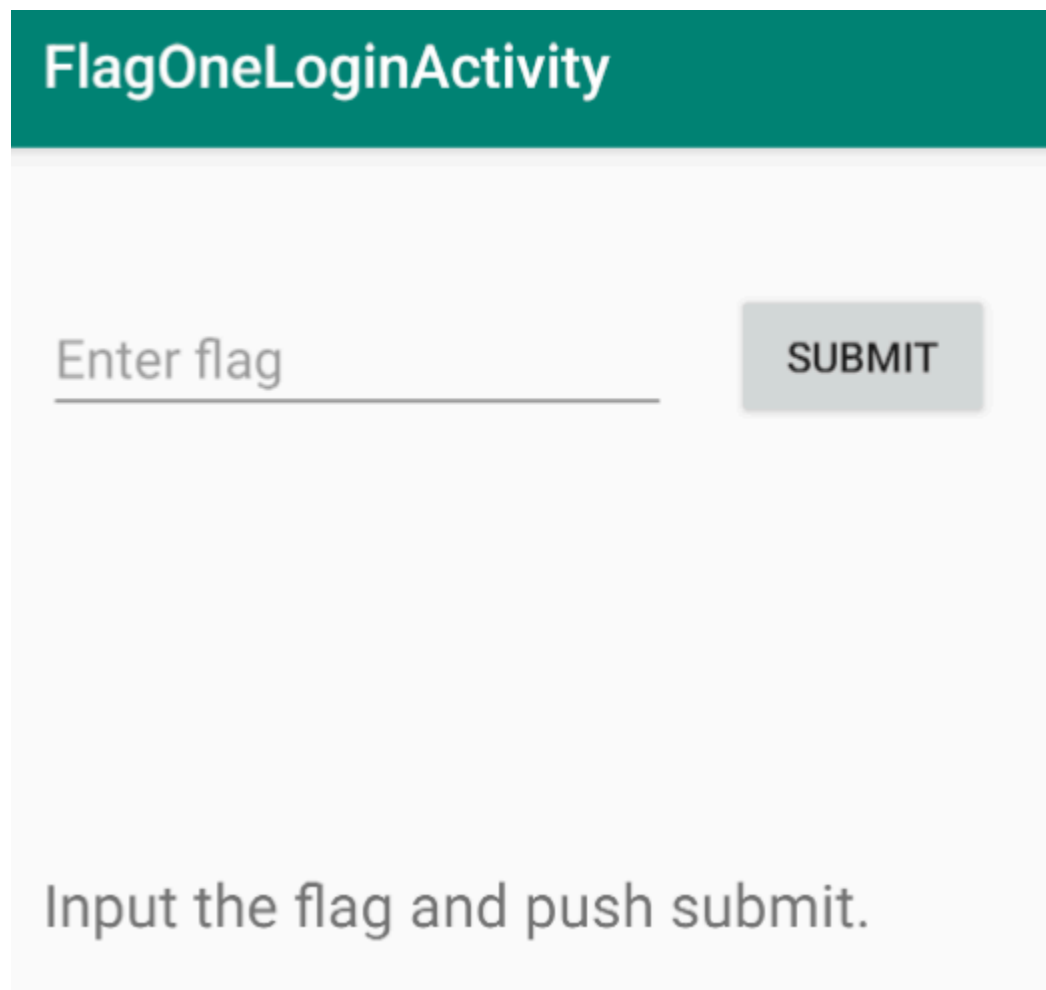
apktools

jadx

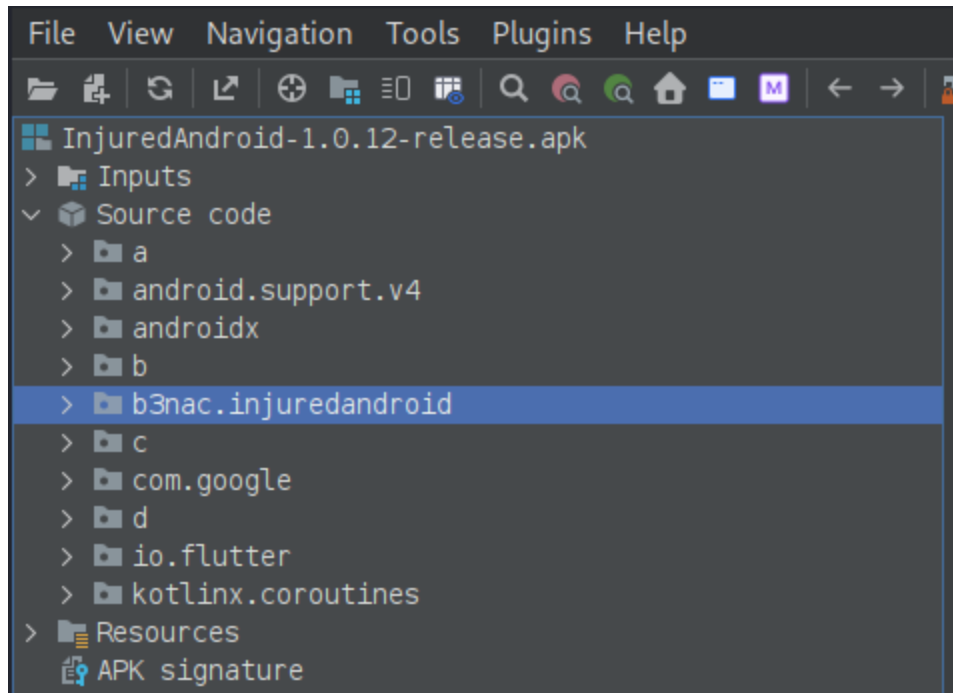
genymotion

frida for dynamic mobile app testing

Flag One

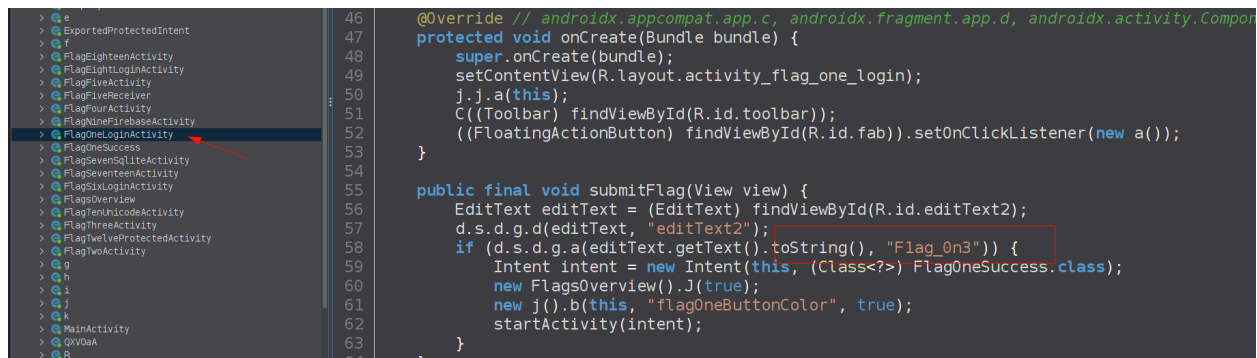


The screenshot shows the 'FlagOneLoginActivity' interface. It features a teal header with the title 'FlagOneLoginActivity'. Below the header is a light gray area containing a text input field with the placeholder text 'Enter flag' and a gray 'SUBMIT' button. At the bottom of the screen, there is a text prompt: 'Input the flag and push submit.'



open the injured application in jadx and navigate to source code then open b3nac.injuredandroid directory

then open the activities of each flag



**Flag\_2**

## FlagTwoActivity

There is a way to  
bypass the main  
activity and invoke  
other activities that  
are exported.

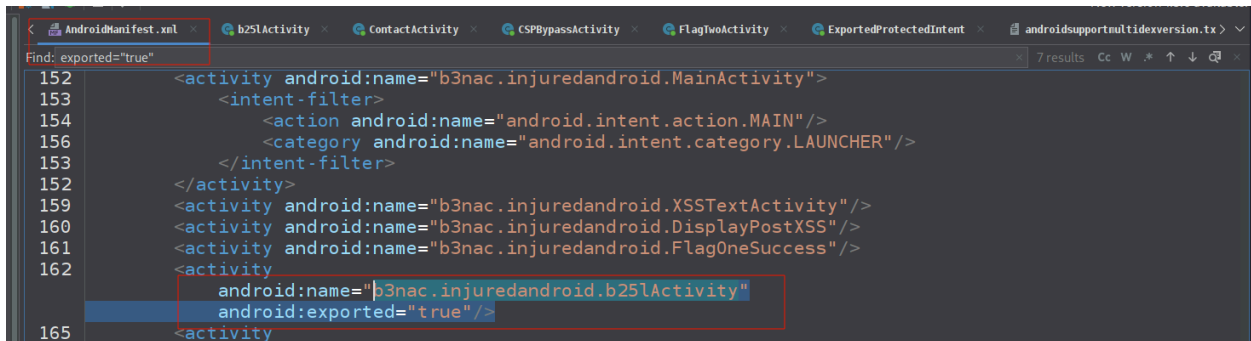
to get the flag two we use the hint that is given here . flagtwo activity is exported so we can bypass or we can execute this activity with external tool since it is exported

we use `adb` to bypass this activity . first

Exported activities are located inside the AndroidManifest.xml file of an Android application.

This file is a structural map of the app. It defines all components, including activities, services, broadcast receivers, and content providers

let us see the androidmanifest.xml file of this application

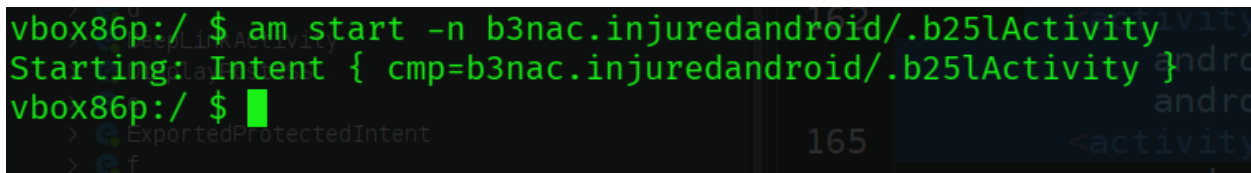


so by executing that activity using external tool we can get the flag 2 when it launched from adb shell

"The component `b3nac.injuredandroid.b25lActivity` is explicitly exported (`android:exported="true"`) without any protective permissions. Utilizing an external testing framework (ADB), an attacker or a malicious third-party application residing on the device can dynamically invoke this activity to bypass intended business logic or access restricted application states."

we use the following command to execute the activity in adb shell

```
am start -n b3nac.injuredandroid/.b25lActivity
```

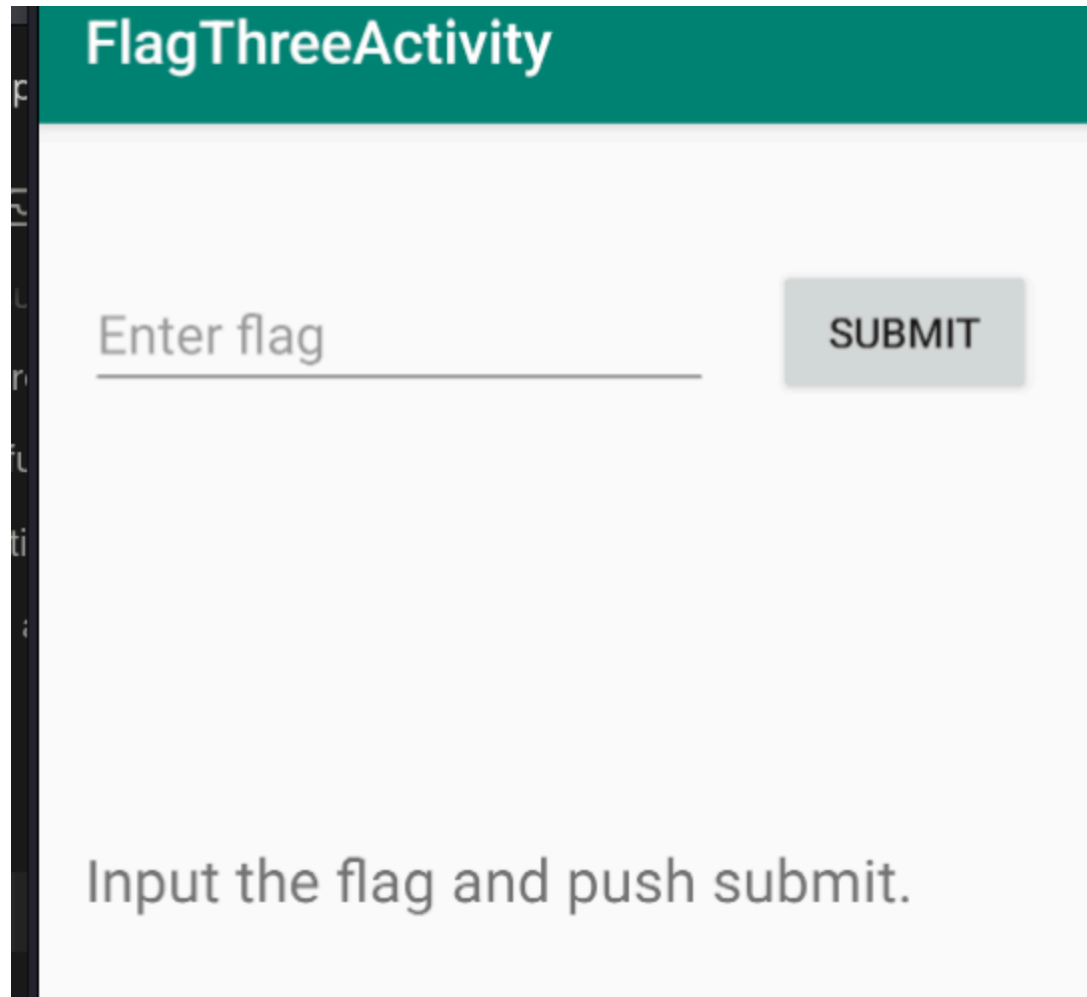


after running this command observe the flag2 is appearing on the android screen

InjuredAndroid

S3c0nd\_F1ag

Flag\_3



according to the hint it is saying flag3-resources

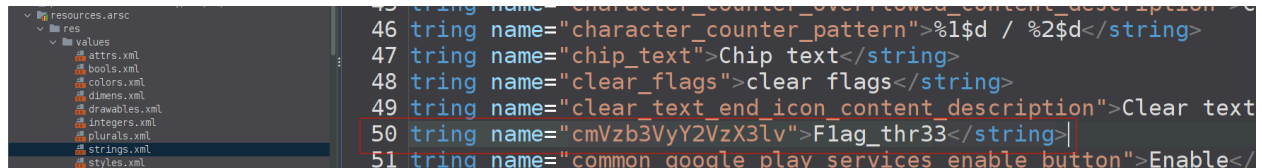
so we follow the hint from the source code that is check .xml files

then below the above function we also got the hint function to get the flag

```
public final void submitFlag(View view) {  
    EditText editText = (EditText) findViewById(R.id.editText2);  
    d.s.d.g.d(editText, "editText2");  
    if (d.s.d.g.a(editText.getText().toString(), getString(R.string.cmVzb3VyY2VzX3lv))) {  
        Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);  
        new FlagsOverview().L(true);  
        new j().b(this, "flagThreeButtonColor", true);  
        startActivity(intent);  
    }  
}
```

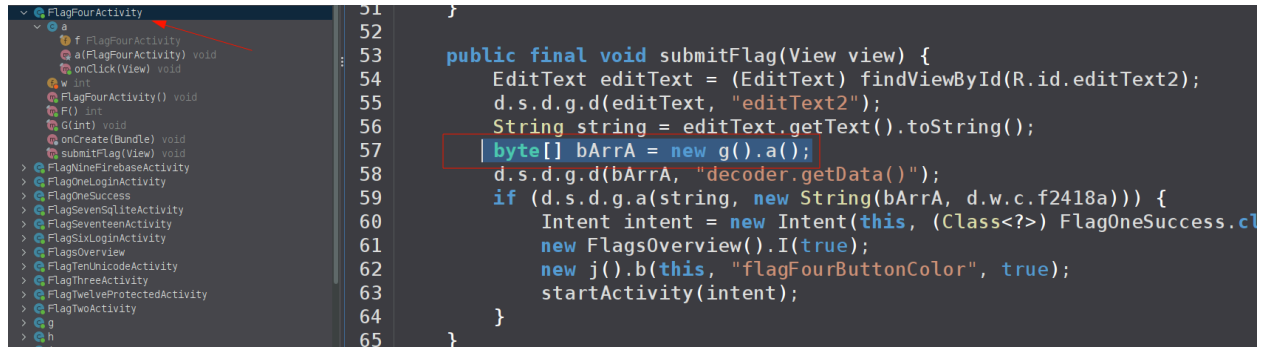
in this function we see the base64 encoded text `cmVzb3VyY2VzX3lv = resources_yo`

then navigate to resource directory  $\Rightarrow$  resource.arsc  $\Rightarrow$  res  $\Rightarrow$  values  $\Rightarrow$  strings.xml



## flag\_4

go to FlagFourActivity class and observe the code

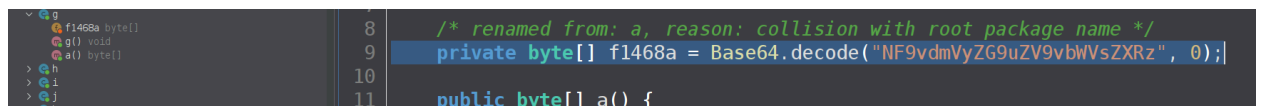


`byte[] bArrA = new g().a();`

this line is the exact hint we want to get the flag for the fourth flag

`byte[] bArrA = new g().a();` means the application is creating a new instance of an obfuscated class named `g` and executing its method named `a()` to retrieve the raw bytes of the secret flag

so navigate to class `g()` on the source page



decode that base64 it gives the fourth flag

## Flag\_5

## FLAG FIVE - EXPORTED BROADCAST RECEIVER

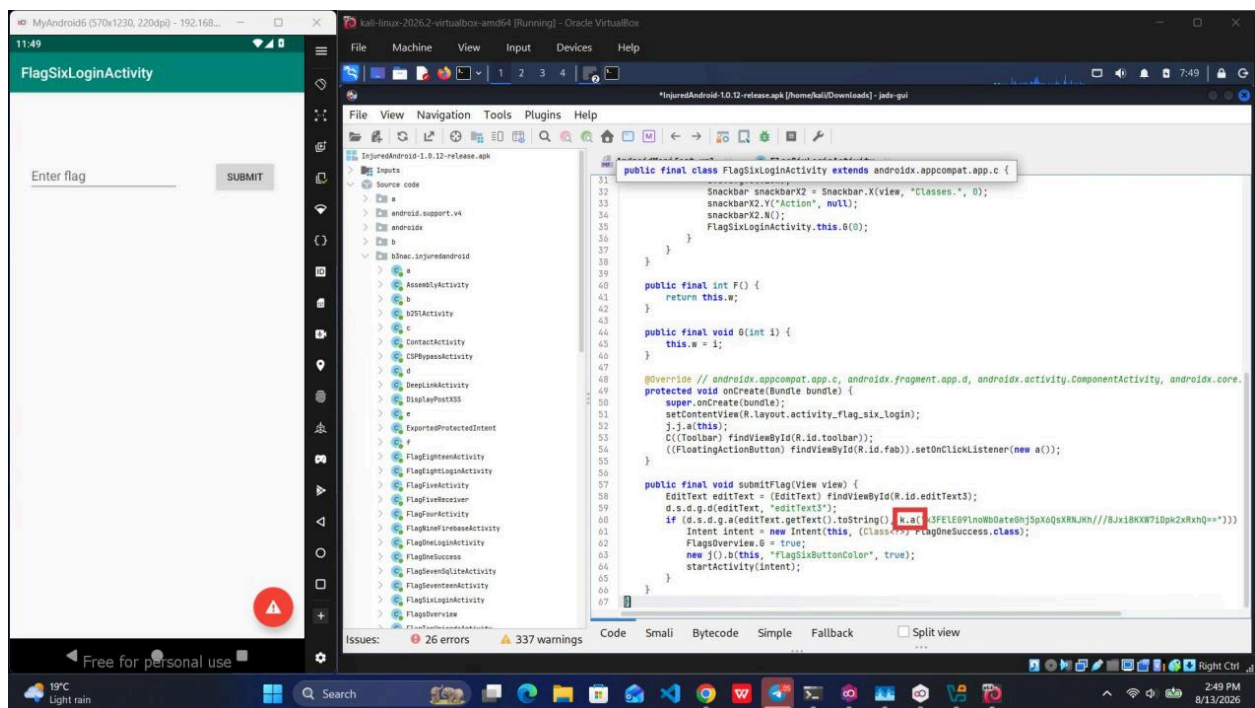
to solve this flag just open 3 times this challenge page then the job done

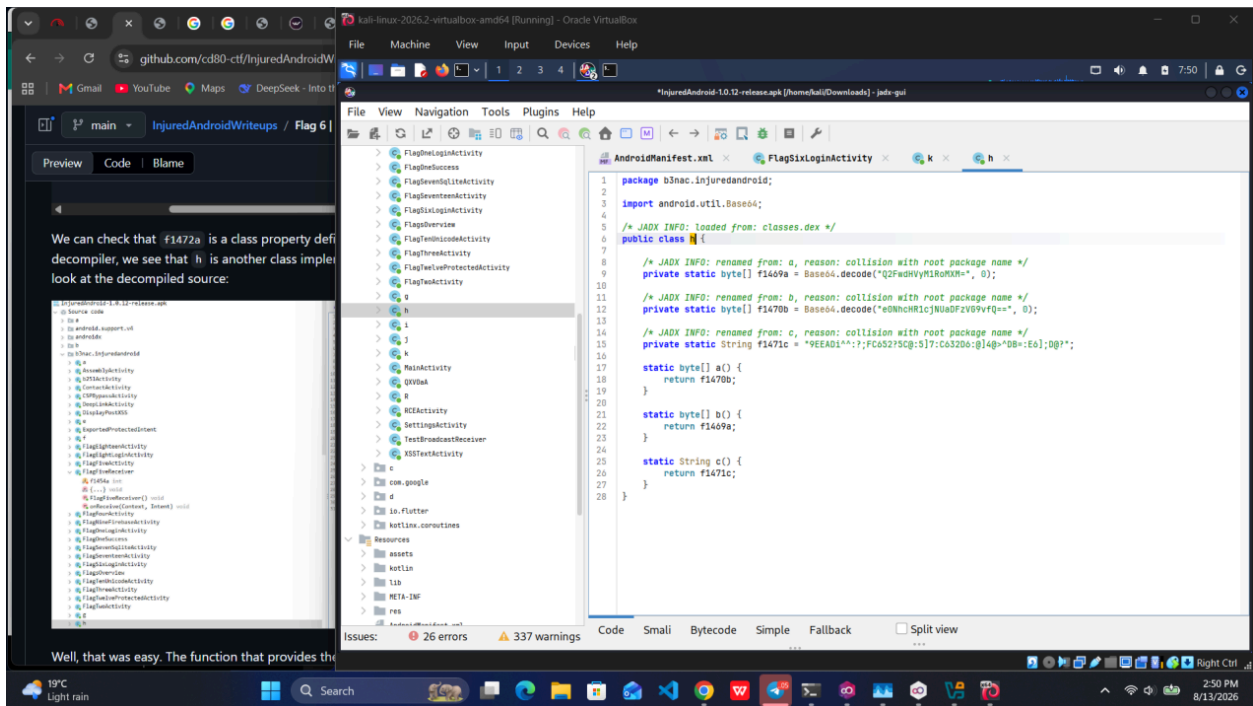
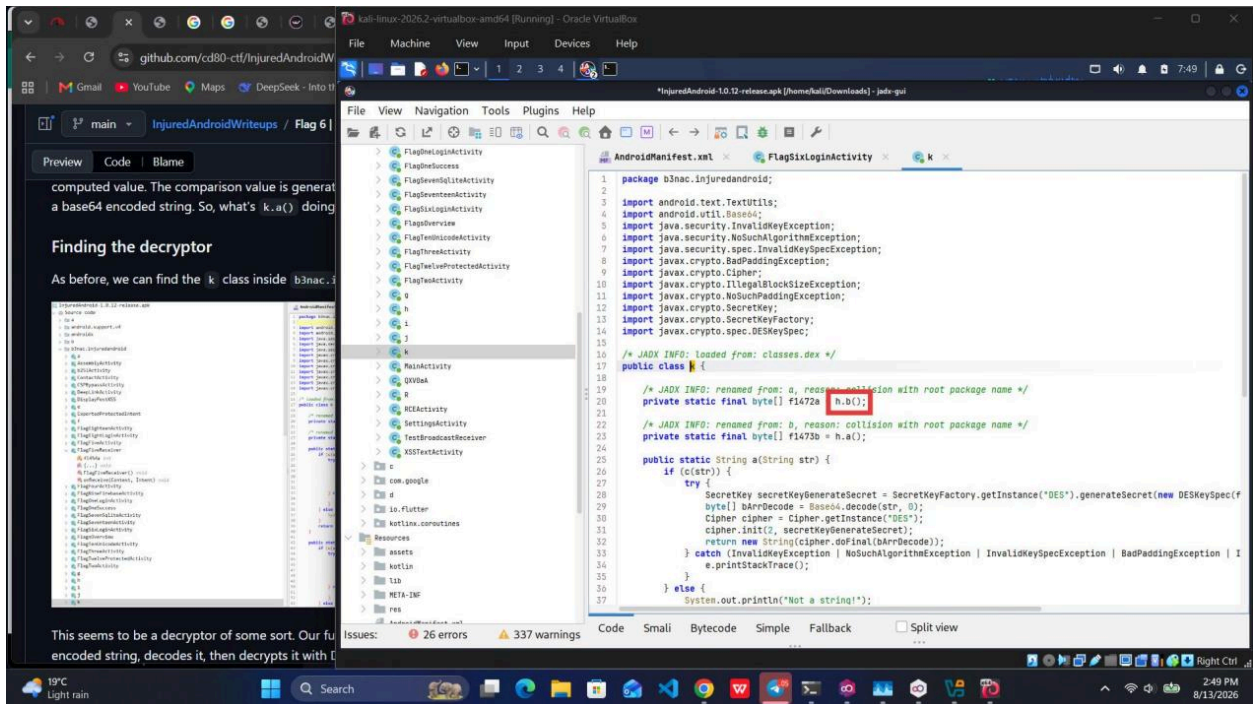
### Flag\_6

### Exploitation steps

1.

open the flagsixloginactivity which shows a k class





1. In the `h` class I get a base64 Encrypted key  
 Then decrypt it

**Flag/solution:**{This\_Isn't\_Where\_I\_Parked\_My\_Car}

## **Impact & remediation**

### **Impact**

- Hardcoded Cryptographic Keys
- Use of Deprecated/Weak Encryption Algorithms (DES)
- Lack of Runtime Integrity Controls & Frida Detection

### **Remediation**

- Secure Key Storage (Android Keystore)
- Upgrade to Strong Encryption (AES-256-GCM)
- Implement Anti-Tampering & Emulator/Root Detection

## **1. Flag Eight- AWS**

### **Methodology & analysis**

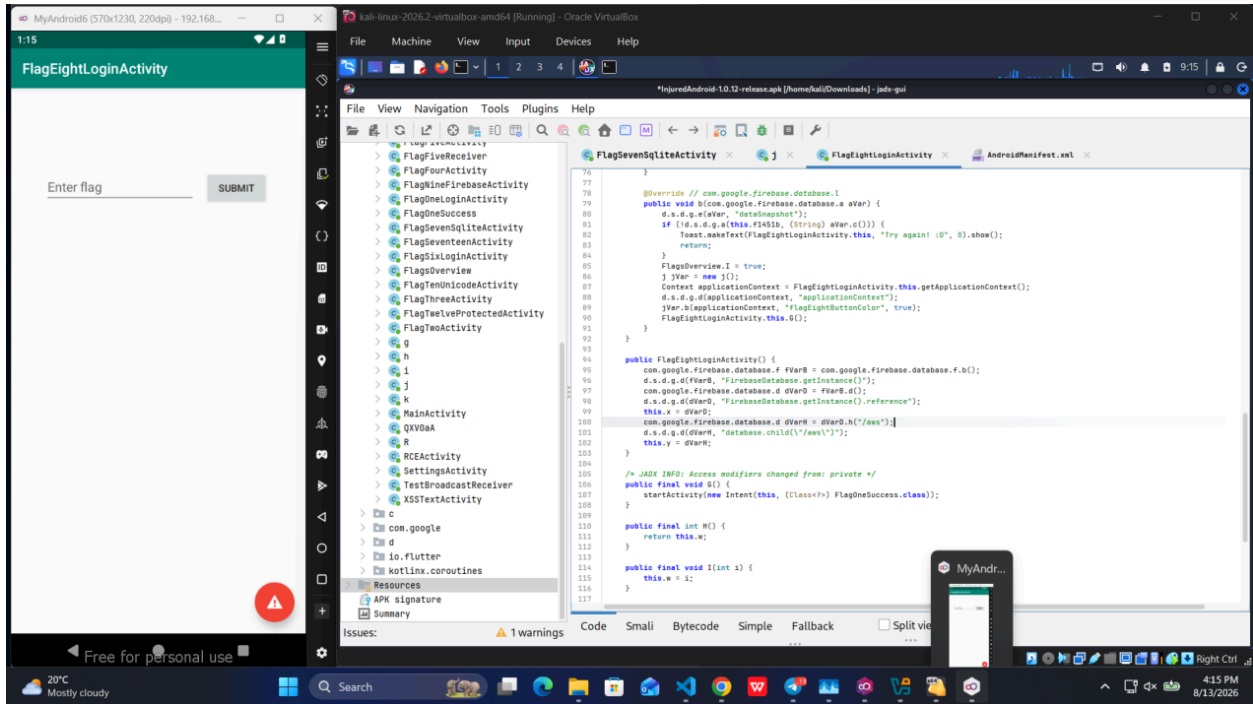
This app shows a input box and a submit button.

Tool used: jadx, Genymotion, browser

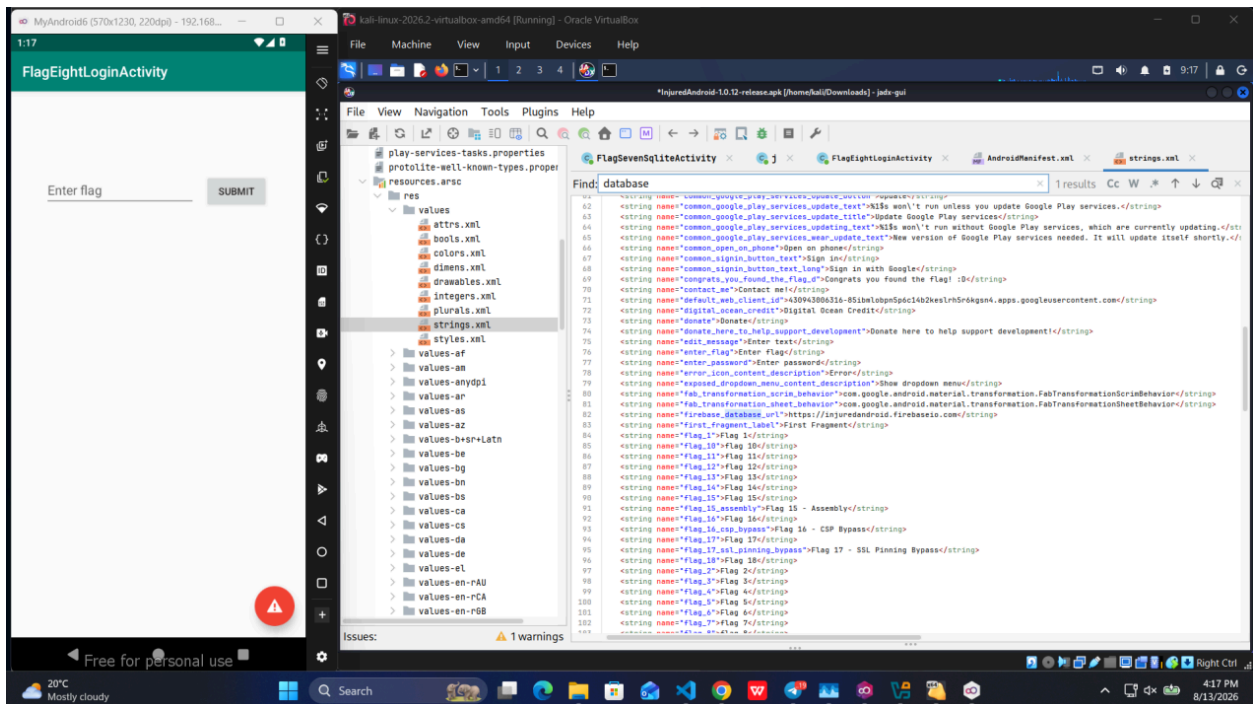
I check the FlagEightActivityclass and find the vulnerabilities

### **Exploitation steps**

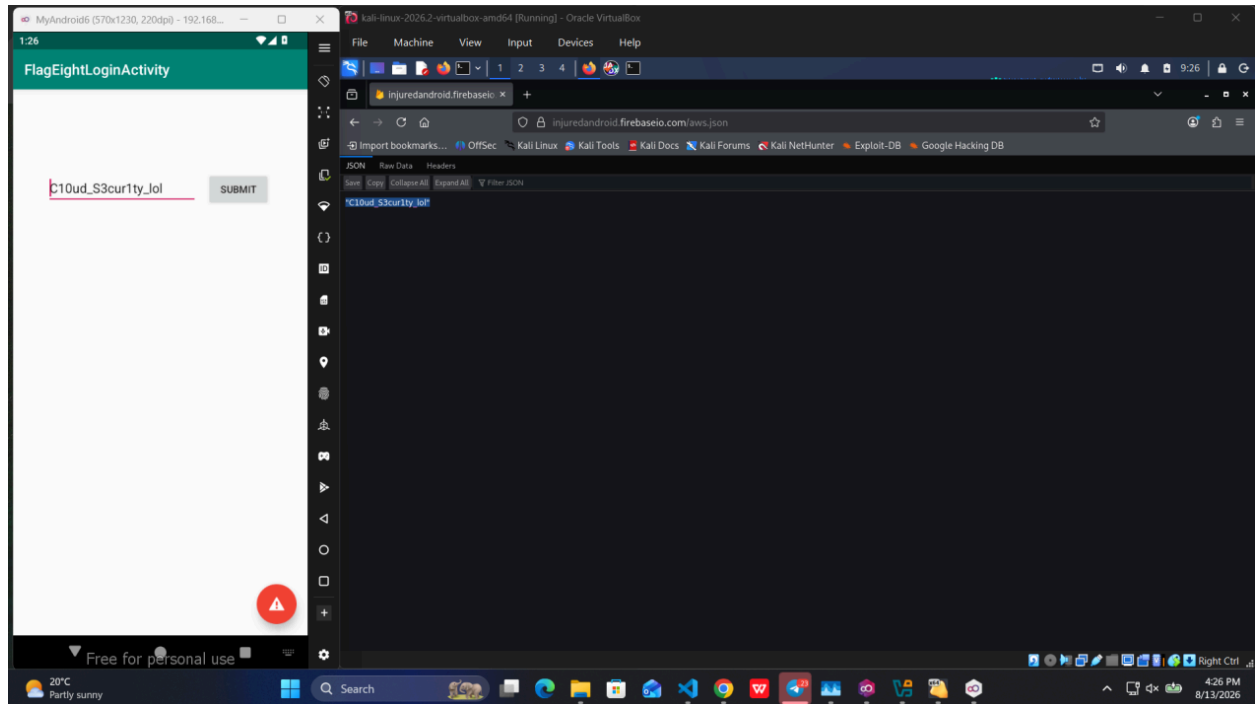
1. I open FlagEightLoginActivity class



2 Then I opened the strings.xml file and find the word “database” which gave me a link to a website.



3 Then we get a string in the website.



2 Submit and complete the task

